

Bocconi University - 20236 Time Series Analysis

Take home assignment 1

Artūrs Jānis Ņikitins, Artoni Emanuele - Group 27

2026-02-06

****EXERCISE 1****

****EXERCISE 1.1****: generate a random sample of size 24 from a $N(0, 2)$ distribution, and pretend it is a time series. The following printout shows the generated time series and its characteristics.

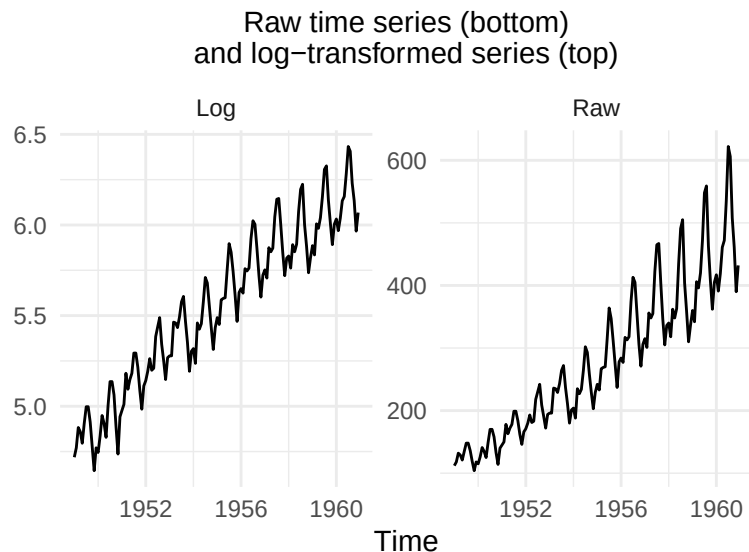
```
## Time Series:
## Start = 1
## End = 24
## Frequency = 1
## [1]  0.9981535  1.0581838 -0.0198699  0.1052981  0.5320154 -1.0929419
## [7] -0.6355229  0.6948973  1.0975490  1.6099245 -0.6577780  4.4209933
## [13]  0.6575613  0.2838283  0.5561364  0.1500619  1.3693263  1.6018915
## [19]  1.6606316  2.1496453 -1.0157562  1.4302392  2.8156475 -1.6744814
```

****EXERCISE 1.2****: reduce the size of ggplot output.

Given the code chunk included in the LAB1 document, we reduce the plot's size by modifying the chunk's header to include "fig.width = 5, fig.height = 3".

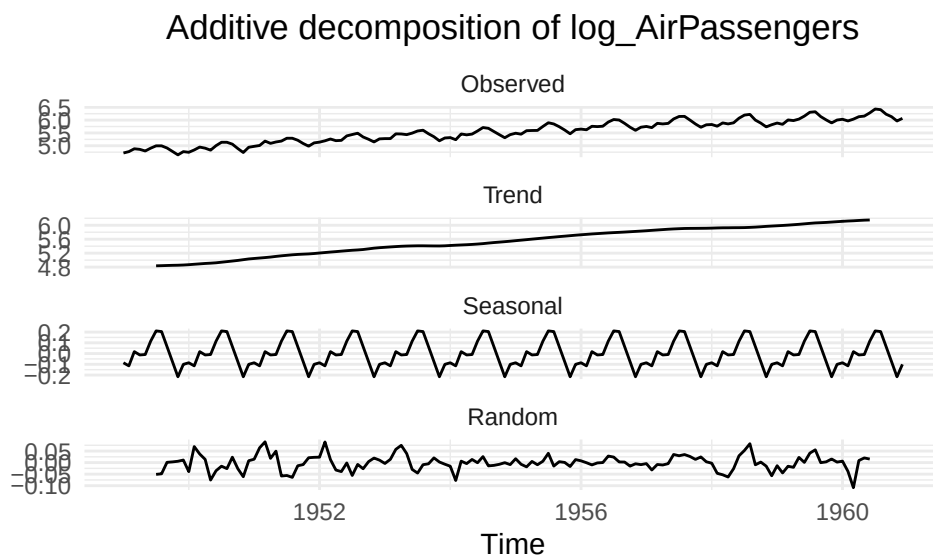
****EXERCISE 1.3****: What does the function `stl` do, what techniques does it use? Explain STL stands for Season and Trend decomposition using Loess. Given a univariate time series, it identifies a seasonal, trend and remainder components, assuming an additive decomposition. LOESS estimates a smooth curve by fitting local regressions around each point, weighting nearby observations more heavily than distant ones. The algorithm proceeds iteratively through inner and outer loops. In the inner loop, the seasonal and trend components are alternately updated using LOESS smoothing. In the outer loop, robustness weights are recomputed to reduce the influence of outliers; these updated weights are then used in the next inner-loop iteration. STL first applies loess smoothing to detrend the series; then, it applies the same method separately to each seasonal subseries (e.g. to all quarters). The remainder is thus deprived of any structure. STL differs from classical time series decomposition methods, which typically involve moving averages to estimate trends and assume seasonal patterns to be constant, in that it allows for flexibility in specifying the amounts of variation in the trend and seasonal components.

****EXERCISE 1.4****:



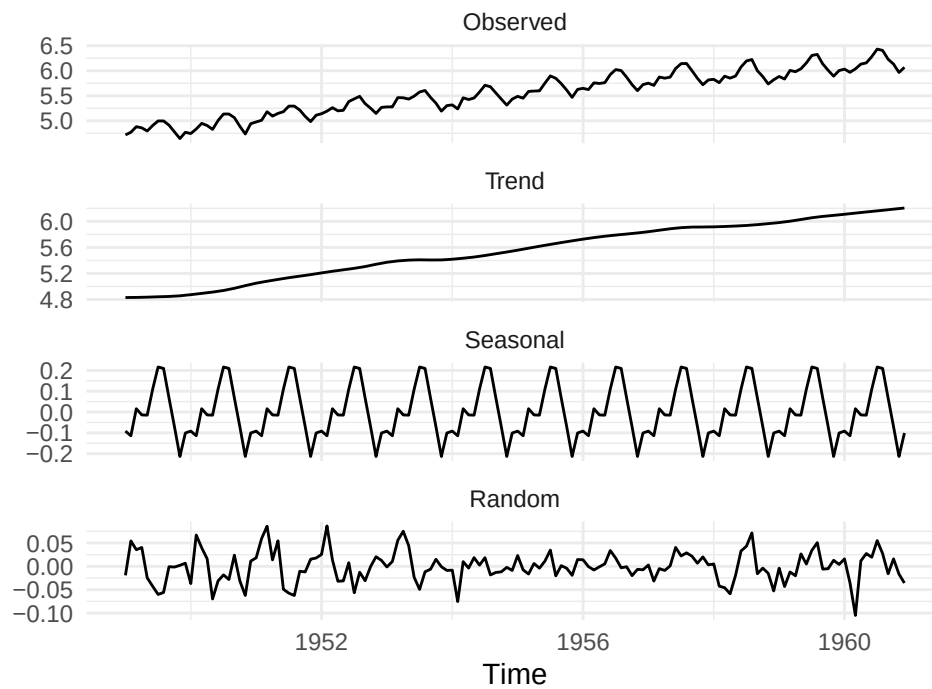
By comparing the dataset to its log-transformation, it appears obvious that the series is heteroskedastic, violating stationarity. We can then conveniently model the series by taking the log-transform and then fitting an additive decomposition. This yields the result included in the following plot.

```
## Warning: Removed 24 rows containing missing values or values outside the scale range
## (`geom_line()`).
```



It is also possible to take logs and apply an additive decomposition using loess. In particular, with `s.window = "periodic"`, the method forces the seasonal patterns to be constant, making it closer to classical decomposition. However, the smoothed trend appears more gradual. The decomposition's results are plotted in the following figure.

STL decomposition of log_AirPassengers

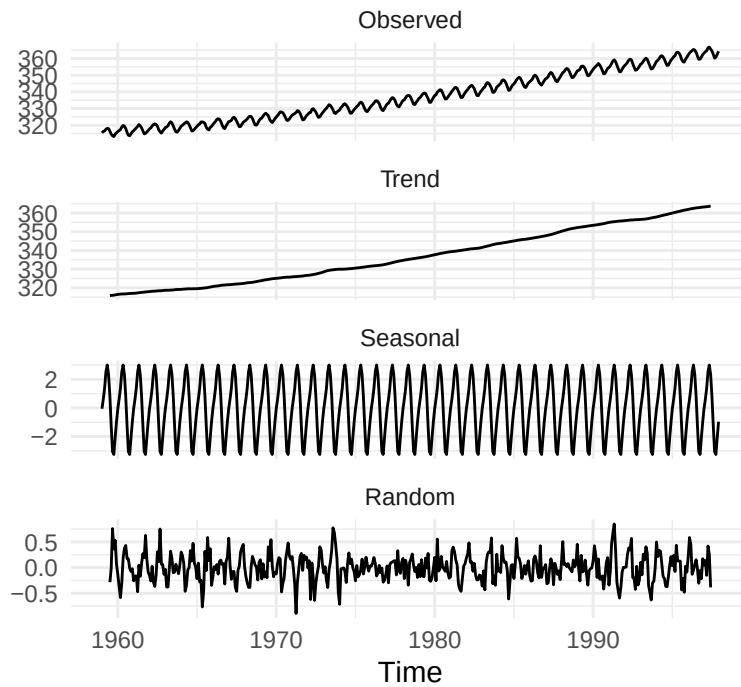


****EXERCISE 2****

The following figures contain the classical decomposition of the time series and a comparison of the detrended and stationary series, and of the residuals.

```
## Warning: Removed 24 rows containing missing values or values outside the scale range
## (`geom_line()`).
```

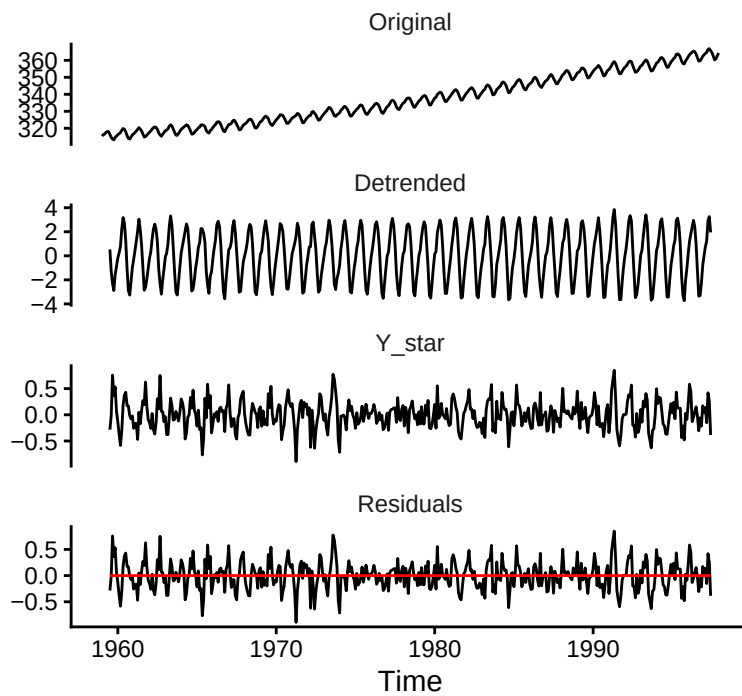
Additive decomposition of CO2



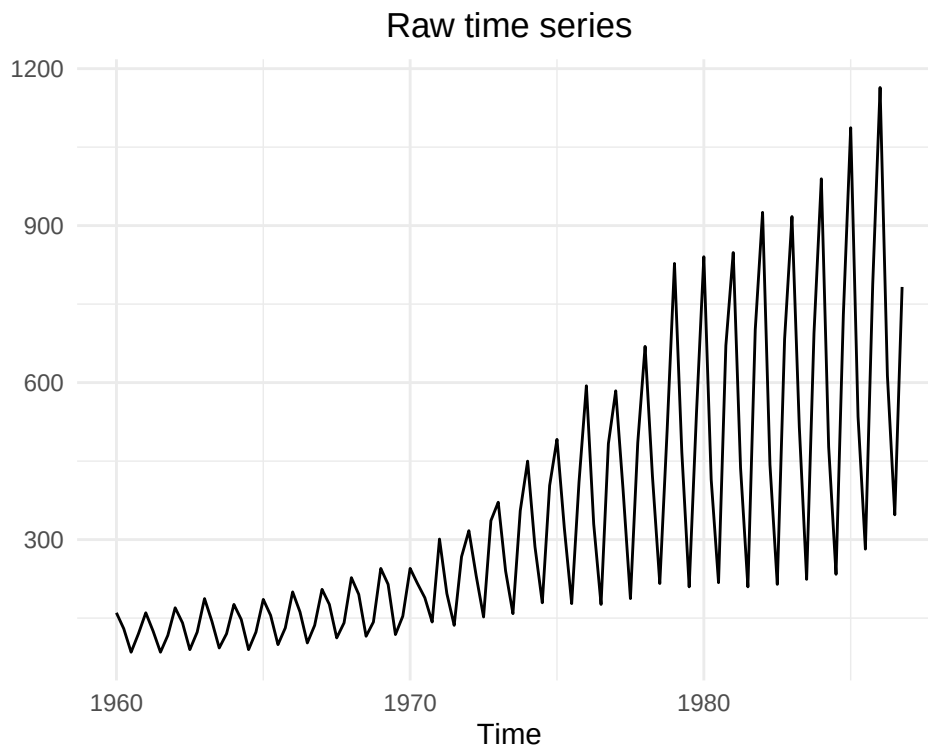
```
## Warning: Removed 36 rows containing missing values or values outside the scale range  
## (`geom_line()`).
```

```
## Warning: Removed 12 rows containing missing values or values outside the scale range  
## (`geom_line()`).
```

CO2: original, detrended, Y^* , residuals with Y^* in red

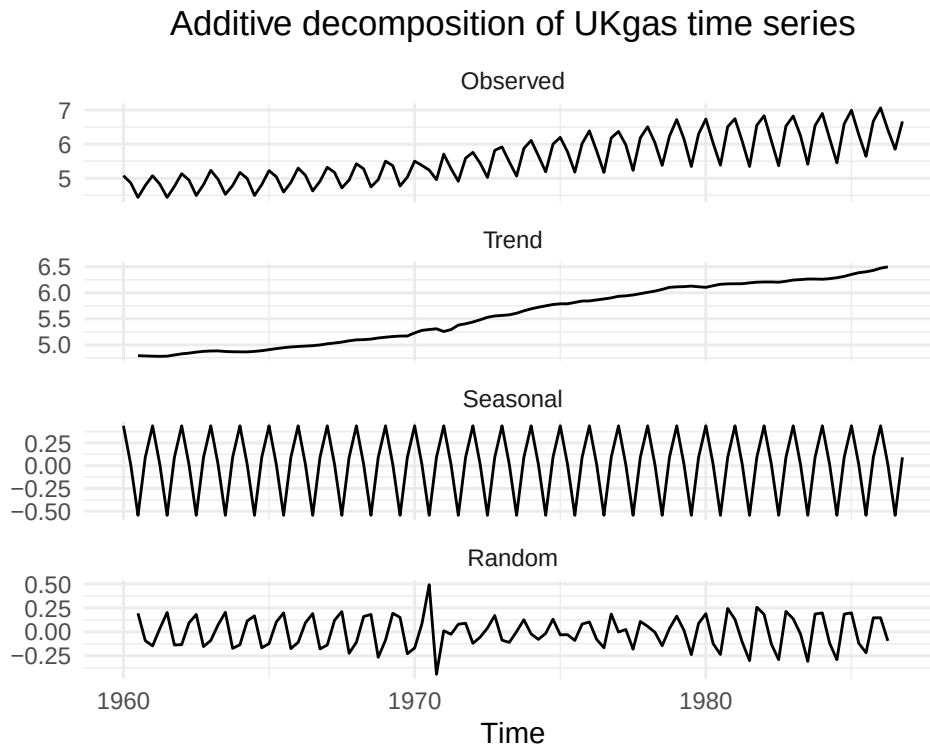


****EXERCISE 3****

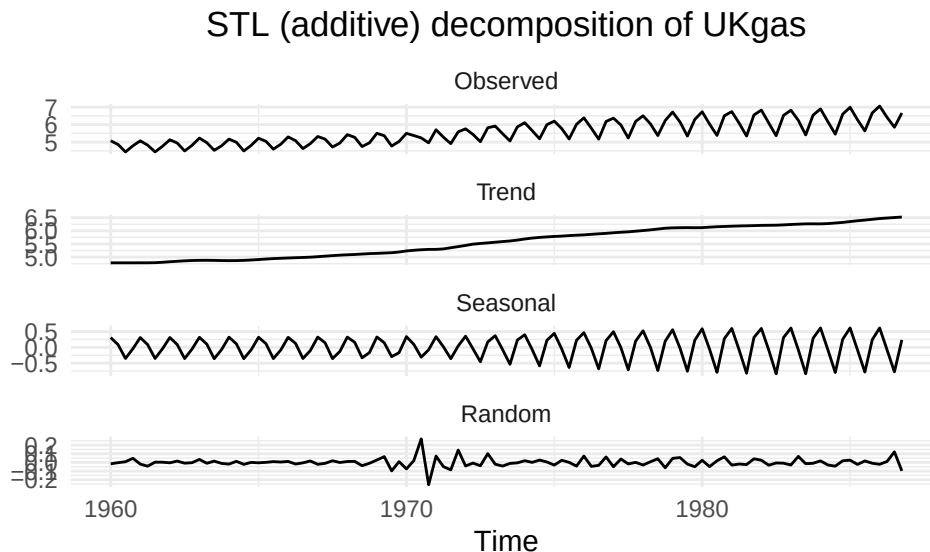


Warning: Removed 8 rows containing missing values or values outside the scale range

```
## (`geom_line()`).
```

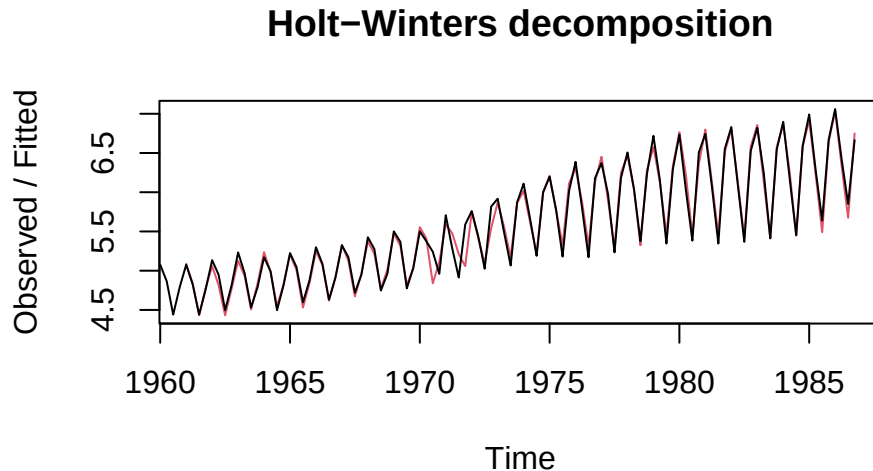


The time series is clearly heteroskedastic. It is possible to either use a multiplicative decomposition, or to take the log-transform of the time series to stabilize the variance, and then apply an additive decomposition. In the results, we have adopted this latter approach.



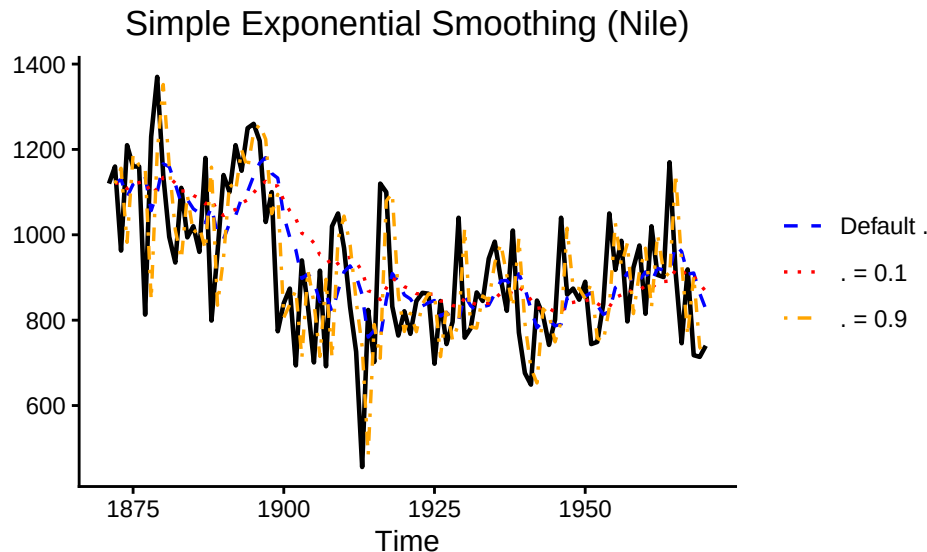
The classical additive decomposition shows that the residuals' distribution over time is non-random around 1970, suggesting that the model's assumption of constant seasonal amplitude is violated. In particular, since classical decomposition constrains the seasonal component to be time-invariant, any change in seasonal pattern is absorbed by the residuals. To confirm and address this issue, we can run an alternative STL decomposition to the same log-transformed series, a method which allows the seasonal component to vary over time. We

can thus clearly determine that the seasonal amplitude indeed changes over time, and that the rate of change also increases significantly after the early 1970's. This is an indication that the classical time series decomposition is inadequate at identifying the evolving seasonal pattern.



After applying an additive model to the log-transformation of the series, the fitted Holt-Winters procedure tracks both the trend and the seasonal variation, indicating that a level-trend-seasonality exponential smoothing procedure fits the data well. In particular, the decomposition fixes $\alpha = 0.029$, $\beta = 1$, $\gamma = 0.725$.

****EXERCISE 4****

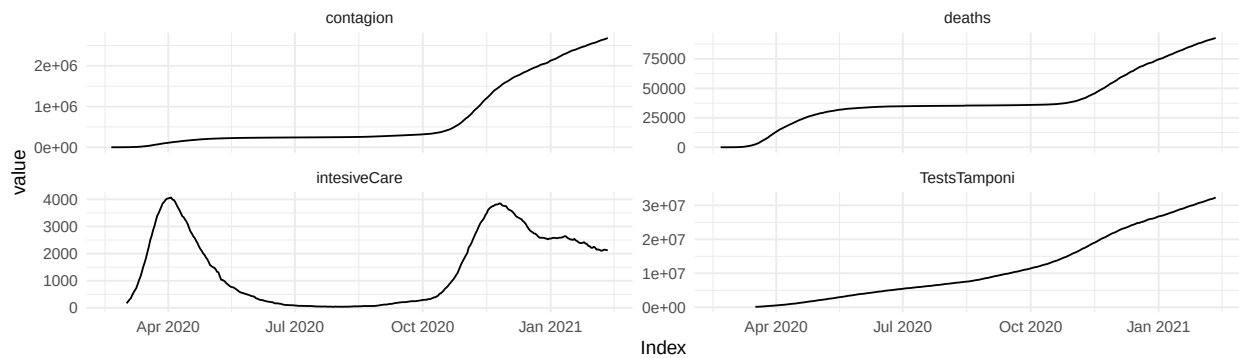


For $\alpha = 0.2466$, the MAPE is 0.1307; for $\alpha = 0.1$, the MAPE is 0.1348; for $\alpha = 0.9$, the MAPE is 0.1450. Using Holt-Winters with $\beta = \gamma = 0$, the estimated α is 0.2466. As the smoothing parameter, α controls the responsiveness of the level estimate. When $\alpha = 0.1$, the prediction depends on many earlier values, making for stable and slow predictions. When $\alpha = 0.9$, the model is highly adaptive, reacting strongly to the most recent observation. Thus, forecasts adjust quickly,

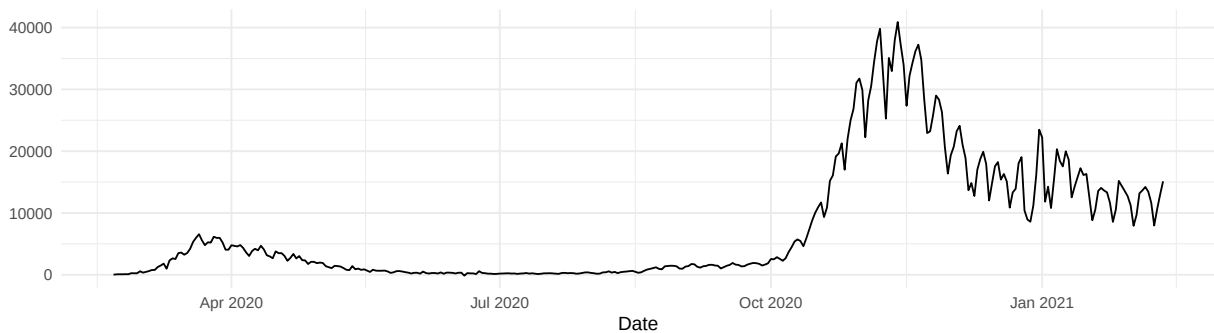
but are also susceptible to noise. Therefore, it makes sense that the value chosen by the model to minimize the loss function is a compromise between the two. In fact, the drop observed at the end of the recorded series is captured most strongly by setting $\alpha = 0.9$, and least so by setting $\alpha = 0.1$, which yields the highest predicted value. In particular, the MAPE is lowest for $\alpha = 0.2466$; highest for $\alpha = 0.9$, confirming that the model's chosen α maximizes predictive performance.

****EXERCISE 5****

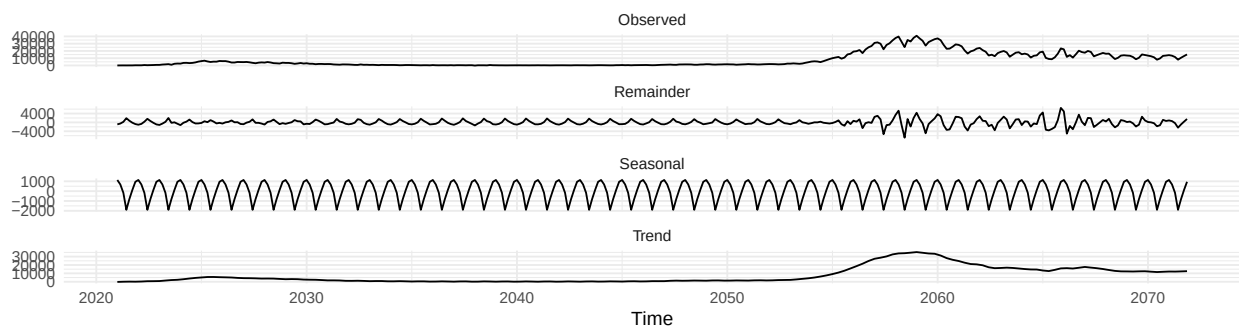
```
## Warning: Removed 38 rows containing missing values or values outside the scale range
## (`geom_line()`).
```

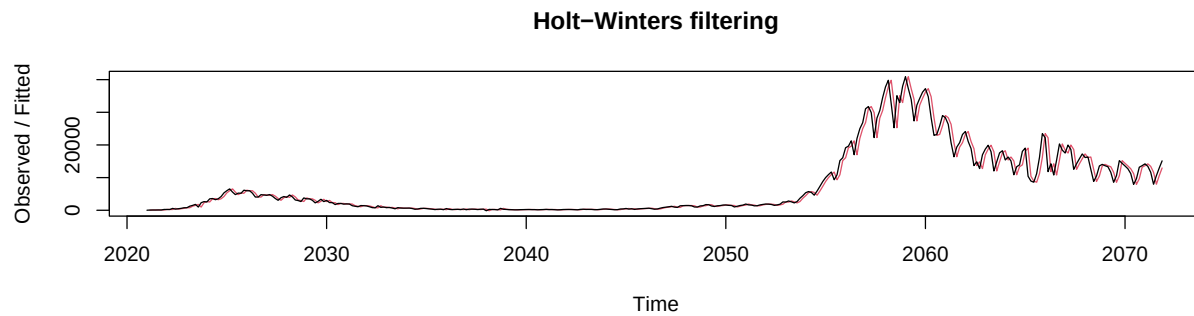


Daily new cases



STL decomposition of daily new cases





```
## [1] 0.9999339
```

The raw data evidences an upward trend in both cumulative cases (contagion) and deaths, and strong growth in testing over time. It is also clear that intensive care facilities experienced epidemic waves. Since the contagion and deaths measures accumulate flows, we have constructed a time series including daily new cases and analyzed it. Graphically, this series displays clear epidemic waves, in particular in the later months of 2020. STL decomposition confirms the presence of a trend component and a clear weekly seasonal component, which we believe to be consistent with reporting effects. Exponential smoothing via Holt-Winters yields a smoothing parameter $\alpha = 0.999$, implying strong persistence and near-random walk behaviour in the short run. Overall, all the series are non-stationary and characterized by wavy effects.

Appendix: All code for this report

```
library(ggplot2)
library(tidyr)
library(tidyverse)
library(tsibble)
library(xts)
library(ggfortify)
library(dplyr)
knitr::opts_chunk$set(echo = FALSE)

library(ggplot2)
library(tidyr)

setwd("~/Documents/R")

plot_ts_gg <- function(x,
                        title = NULL,
                        theme_fn = theme_bw,
                        facet_ncol = 1,
                        wrap_width = 45) {

  if (inherits(x, "decomposed.ts")) {

    df <- data.frame(
      Time      = as.numeric(time(x$x)),
      Observed = as.numeric(x$x),
```

```

    Trend    = as.numeric(x$trend),
    Seasonal  = as.numeric(x$seasonal),
    Random    = as.numeric(x$random)
  )
} else if (inherits(x, "stl")) {

  df <- data.frame(
    Time      = as.numeric(time(x$time.series)),
    Observed  = as.numeric(rowSums(x$time.series)),
    Trend     = as.numeric(x$time.series[, "trend"]),
    Seasonal  = as.numeric(x$time.series[, "seasonal"]),
    Random    = as.numeric(x$time.series[, "remainder"])
  )

} else if (inherits(x, "ts")) {

  df <- data.frame(
    Time = as.numeric(time(x)),
    Value = as.numeric(x)
  )

  p <- ggplot(df, aes(Time, Value)) +
    geom_line() +
    labs(title = title, x = "Time", y = NULL) +
    theme_fn() +
    theme(plot.title = element_text(hjust = 0.5))

  if (!is.null(title)) {
    p <- p + labs(title = paste(strwrap(title, width = wrap_width),
                                collapse = "\n"))
  }

  return(p)
} else {
  stop("Input must be ts, decomposed.ts, or stl object.")
}

# Decomposition branch (decompose or stl)
df_long <- pivot_longer(df, cols = -Time,
                        names_to = "Component",
                        values_to = "Value")

df_long$Component <- factor(
  df_long$Component,
  levels = c("Observed", "Trend", "Seasonal", "Random")
)

p <- ggplot(df_long, aes(Time, Value)) +
  geom_line() +
  facet_wrap(~ Component, ncol = facet_ncol, scales = "free_y") +
  labs(title = title, x = "Time", y = NULL) +

```

```

    theme_fn() +
    theme(
      plot.title = element_text(hjust = 0.5),
      strip.background = element_blank()
    )

    if (!is.null(title)) {
      p <- p + labs(title = paste(strwrap(title, width = wrap_width),
                                collapse = "\n"))
    }

    p
  }

plot_multi_ts_gg <- function(x,
                             title = NULL,
                             theme_fn = theme_bw,
                             wrap_width = 45) {

  if (!inherits(x, "ts"))
    stop("x must be a ts or multivariate ts (e.g., cbind(...))")

  df <- data.frame(Time = as.numeric(time(x)),
                   as.data.frame(x))

  df_long <- pivot_longer(df, cols = -Time,
                          names_to = "Series",
                          values_to = "Value")

  if (!is.null(title))
    title <- paste(strwrap(title, width = wrap_width), collapse = "\n")

  ggplot(df_long, aes(Time, Value)) +
    geom_line() +
    facet_wrap(~ Series, ncol = 1, scales = "free_y") +
    labs(title = title, x = "Time", y = NULL) +
    theme_fn() +
    theme(
      plot.title = element_text(hjust = 0.5),
      strip.background = element_blank()
    )
}

# Exercise 1.1
x<-rnorm(24, sd=2)
y<-ts(x)
y

# Exercise 1.4
log_AirPassengers<-log(AirPassengers)
comb<-cbind(log_AirPassengers,AirPassengers)

```

```

library(ggplot2)
library(tidyr)

df <- data.frame(
  Time = as.numeric(time(AirPassengers)),
  Raw = as.numeric(AirPassengers),
  Log = as.numeric(log(AirPassengers))
)

df_long <- pivot_longer(df, cols = c(Log, Raw),
                        names_to = "Series",
                        values_to = "Value")

df_long$Series <- factor(df_long$Series, levels = c("Log", "Raw"))

ggplot(df_long, aes(Time, Value)) +
  geom_line() +
  facet_wrap(~ Series, ncol = 2, scales = "free_y") +
  labs(
    title = "Raw time series (bottom)\nand log-transformed series (top)",
    x = "Time",
    y = NULL
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 11, hjust = 0.5),
    strip.background = element_blank()
  )

m<-decompose(log_AirPassengers,type="additive")

plot_ts_gg(
  m,
  title = "Additive decomposition of log_AirPassengers",
  theme_fn = theme_minimal
)

s<-stl(log_AirPassengers,s.window="periodic")

plot_ts_gg(
  s,
  title = "STL decomposition of log_AirPassengers",
  theme_fn = theme_minimal
)

# Exercise 2

# decompose and plot the components
m<-decompose(co2,type="additive")

plot_ts_gg(
  m,
  title = "Additive decomposition of CO2",
  theme_fn = theme_minimal
)

```

```

# 4-panel plot
detrended <- co2 - m$trend
Y_star    <- co2 - m$trend - m$seasonal
residuals <- m$random
diff_line <- residuals - Y_star

df <- data.frame(
  Time      = as.numeric(time(co2)),
  Original  = as.numeric(co2),
  Detrended = as.numeric(detrended),
  Y_star    = as.numeric(Y_star),
  Residuals = as.numeric(residuals),
  Diff      = as.numeric(diff_line)
)

# long data for the 4 panels
df_long <- pivot_longer(
  df,
  cols = c(Original, Detrended, Y_star, Residuals),
  names_to = "Panel",
  values_to = "Value"
)

df_long$Panel <- factor(df_long$Panel,
  levels = c("Original", "Detrended", "Y_star", "Residuals"))

# data for the red line
df_diff <- data.frame(
  Time = df$Time,
  Panel = factor("Residuals", levels = levels(df_long$Panel)),
  Diff = df$Diff
)

ggplot(df_long, aes(Time, Value)) +
  geom_line() +
  geom_line(data = df_diff, aes(Time, Diff), color = "red") +
  facet_wrap(~ Panel, ncol = 1, scales = "free_y") +
  labs(title = strwrap("CO2: original, detrended, Y*, residuals with Y~* in red"),
    x = "Time",
    y = NULL) +
  theme_classic() +
  theme(
    plot.title = element_text(size = 10, hjust = 0.5),
    strip.background = element_blank()
  )

# Exercise 3
#1
plot_ts_gg(
  UKgas,
  title = "Raw time series",
  theme_fn = theme_minimal
)

```

```

)

#2
m<-decompose(log(UKgas), type="additive")
plot_ts_gg(
  m,
  title = "Additive decomposition of UKgas time series",
  theme_fn = theme_minimal
)
s <- stl(log(UKgas), s.window = 7)
plot_ts_gg(
  s,
  title = "STL (additive) decomposition of UKgas",
  theme_fn = theme_minimal
)

hw<-HoltWinters(log(UKgas),seasonal = "additive")
plot(hw, main = "Holt-Winters decomposition")

hw<-HoltWinters(log(UKgas),seasonal = "additive")
note <- sprintf("alpha = %.3f, beta = %.3f, gamma = %.3f",
  hw$alpha, hw$beta, hw$gamma)

# Exercise 4
m<-HoltWinters(Nile, beta = FALSE, gamma = FALSE)
s<-HoltWinters(Nile, alpha = 0.1, beta = FALSE, gamma = FALSE)
t<-HoltWinters(Nile, alpha = 0.9, beta = FALSE, gamma = FALSE)

mape = function(y, yhat){ mean(abs(y - yhat)/y) }

y_m <- window(Nile, start = time(Nile)[2])
m_mape <- mape(y_m, m$fitted[,1])
s_mape <- mape(y_m, s$fitted[,1])
t_mape <- mape(y_m, t$fitted[,1])

df <- data.frame(
  Time = as.numeric(time(Nile)),
  Original = as.numeric(Nile)
)

df_fit <- data.frame(
  Time = as.numeric(time(Nile))[-1],
  Default = as.numeric(m$fitted[, "xhat"]),
  Alpha_0.1 = as.numeric(s$fitted[, "xhat"]),
  Alpha_0.9 = as.numeric(t$fitted[, "xhat"])
)

ggplot() +
  geom_line(data = df, aes(Time, Original),
    color = "black", linewidth = 0.8) +
  geom_line(data = df_fit,
    aes(Time, Default, color = "Default ", linetype = "Default "),
    linewidth = 0.6) +

```

```

geom_line(data = df_fit,
          aes(Time, Alpha_0.1, color = " = 0.1", linetype = " = 0.1"),
          linewidth = 0.6) +
geom_line(data = df_fit,
          aes(Time, Alpha_0.9, color = " = 0.9", linetype = " = 0.9"),
          linewidth = 0.6) +
scale_color_manual(values = c("Default " = "blue",
                              " = 0.1" = "red",
                              " = 0.9" = "orange")) +
scale_linetype_manual(values = c("Default " = "dashed",
                                 " = 0.1" = "dotted",
                                 " = 0.9" = "dotdash")) +
labs(title = "Simple Exponential Smoothing (Nile)",
     x = "Time",
     y = NULL,
     color = NULL,
     linetype = NULL) +
theme_classic() +
theme(plot.title = element_text(hjust = 0.5))

# Exercise 5
data_covid <- read.table("coronavirus-data.txt", header=TRUE, fill = TRUE)

dates <- seq.Date(as.Date("2020-02-20"), by = "day", length.out = nrow(data_covid))
xts_data_covid <- xts(data_covid |> dplyr::select(-day), order.by = dates)

df_wide <- xts_data_covid |> fortify.zoo() # transform xts into dataframe, as ggplot works with these
df_plot <- df_wide |>
  pivot_longer(-Index, names_to = "variable", values_to = "value")

ggplot(df_plot, aes(x = Index, y = value)) +
  geom_line() +
  facet_wrap(~ variable, scales = "free_y") +
  theme_minimal()

new_cases <- diff(data_covid$contagion, lag = 1)

df_new <- data.frame(
  date = dates[-1],
  new_cases = new_cases
)

ggplot(df_new, aes(x = date, y = new_cases)) +
  geom_line() +
  labs(
    title = "Daily new cases",
    x = "Date",
    y = "",
  ) +
  theme_minimal()

# ensure lengths match: length(dates) == length(new_cases)

```

```

cases_ts <- ts(new_cases, start = c(2020, 8), frequency = 7)
# (start is optional - use if you want a meaningful time index)
stl_fit <- stl(cases_ts, s.window = "periodic")

stl_ts <- cbind(
  Observed = cases_ts,
  Trend     = stl_fit$time.series[, "trend"],
  Seasonal  = stl_fit$time.series[, "seasonal"],
  Remainder = stl_fit$time.series[, "remainder"]
)
plot_multi_ts_gg(
  stl_ts,
  title = "STL decomposition of daily new cases",
  theme_fn = theme_minimal
)

ses_fit <- HoltWinters(cases_ts, beta = FALSE, gamma = FALSE)
plot(ses_fit)
ses_fit$alpha

```